

# Deep Networks

MLP/FNN to Universal Function Approximation, Depth and Width Efficiency

Introduction to Deep Learning · Class 9

10 August

# Outline

The Multilayer Perceptron, Formally

Universal Approximation, Revisited

How Wide Must a Network Be?

How Much Does Depth Buy?

Counting Capacity

Reading the Theorems

# The Multilayer Perceptron, Formally

# Notation used throughout this course

|                                          |                                 |                                    |                                                     |
|------------------------------------------|---------------------------------|------------------------------------|-----------------------------------------------------|
| $\mathbf{x} \in \mathbb{R}^D$            | input / <b>features</b>         | $\phi_j(\mathbf{x})$               | <b>fixed</b> basis function                         |
| $y, \hat{y}$                             | target, model <b>prediction</b> | $\mathcal{K} \subset \mathbb{R}^D$ | a compact set                                       |
| $\mathbf{z}^{(l)} = h(\mathbf{a}^{(l)})$ | units of hidden layer $l$       | $\mathcal{C}(\mathcal{K})$         | continuous functions on $\mathcal{K}$               |
| $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$     | weights and biases              | $\ f\ _\infty$                     | $\sup_{\mathbf{x} \in \mathcal{K}}  f(\mathbf{x}) $ |
| $L, M$                                   | <b>depth, width</b>             | $\varepsilon$                      | target accuracy                                     |
| $W$                                      | total number of parameters      | $\mathcal{N}_{L,M}$                | functions a network can compute                     |

Symbols follow **Bishop & Bishop (2024)**.  $W$  counts every weight and bias;  $\mathcal{N}_{L,M}$  is the set of functions a network of that shape can compute.

# Where Class 7 left us

- Composition **multiplies** linear pieces; width only **adds** them
- Universal approximation holds in  $D$  dimensions for any non-polynomial activation, via ridge functions
- ReLU networks compute **exactly** the continuous piecewise-linear functions — shallow or deep
- The sawtooth needs  $O(L)$  units deep and  $2^L - 1$  shallow

All of that was about **what** a network can represent. None of it says **how much network** it takes.

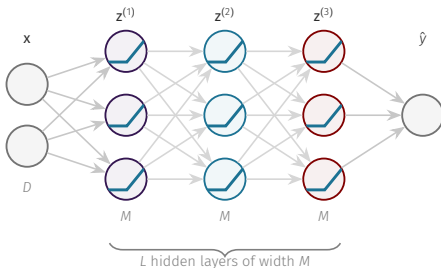
# Three questions, and where they are answered

|   | Question                                           | Answered by                                       |    |
|---|----------------------------------------------------|---------------------------------------------------|----|
| 1 | Can one hidden layer reach any function at all?    | Cybenko, Hornik, Leshno                           | §2 |
| 2 | How wide must a network be before it works at all? | Lu, Pu, Wang, Hu & Wang                           | §3 |
| 3 | How much does depth buy over width?                | Telgarsky; Eldan & Shamir; Montúfar; Serra et al. | §4 |

A fourth question follows from the third: if depth buys so much expressive power, what does that power **cost** in data? That is §5.

# The multilayer perceptron

*Before we can count anything, we need to fix exactly what we are counting.*



$$\mathbf{z}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = h(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}), \quad \hat{\mathbf{y}} = \mathbf{W}^{(L+1)}\mathbf{z}^{(L)} + \mathbf{b}^{(L+1)}$$

Write  $\mathcal{N}_{L,M}$  for the set of functions this computes as the weights range over  $\mathbb{R}^W$ , with

$$W = M(D+1) + (L-1)M(M+1) + (M+1) \text{ parameters.}$$

# Why not just use fixed basis functions?

*Before calling learned features progress, see what goes wrong without them.*

Tile the input space, one basis function per cell (B&B, 2024, §6.1):

$$\hat{y}(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x}), \quad \phi_j \text{ centred on cell } j$$

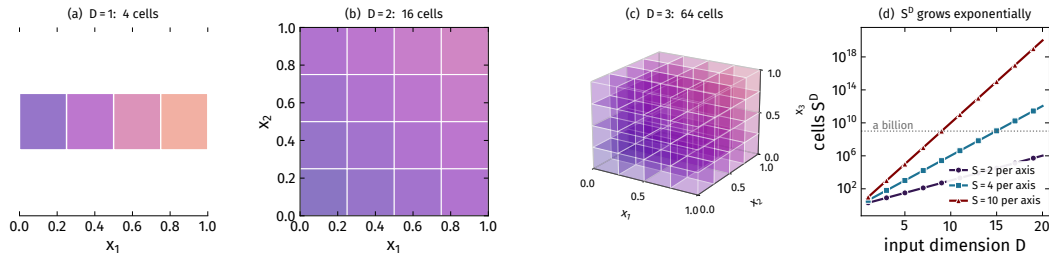
## Proposition 1 (The grid blows up)

*Partition each of the  $D$  axes into  $S$  intervals. The tensor-product grid then has  $S^D$  cells, so the model has  $S^D$  parameters — exponential in the input dimension.*

*Proof.* A cell is one interval chosen per axis, so there are  $|\{1, \dots, S\}^D| = S^D$ . □

With  $S = 10$  and a  $32 \times 32$  image ( $D = 1024$ ):  $10^{1024}$  cells.

# The curse of dimensionality, drawn

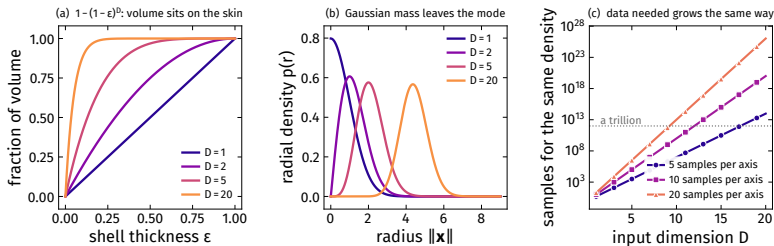


(a–c) a grid with  $S = 4$  cells per axis in one, two and three dimensions: 4, 16, 64 cells. (d) the count  $S^D$  against dimension, at three resolutions.

Every cell needs its own parameter — and its own data to fit it.

# Two more faces of the same problem

*High-dimensional space is strange in two further ways that matter later.*



(a) the volume within  $\epsilon$  of a unit ball's surface,  $1 - (1 - \epsilon)^D$ . (b) a Gaussian's mass migrates to a shell at radius  $\approx \sqrt{D}$ . (c) matching a sample density costs exponentially more data.

**SHALLOW** fixed  $\phi_j$ : one parameter per cell,  $S^D$  of them

**DEEP** learned features: units *move* to where the data is

# Universal Approximation, Revisited

# What “universal” means

*Networks are supposed to escape this. First, say precisely what that claim asserts.*

Fix a compact  $\mathcal{K} \subset \mathbb{R}^D$  and use the supremum norm  $\|f\|_\infty = \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x})|$ .

## Definition

A family  $\mathcal{F} \subset C(\mathcal{K})$  is **universal** (dense) if for every  $f \in C(\mathcal{K})$  and every  $\varepsilon > 0$  there is  $g \in \mathcal{F}$  with  $\|f - g\|_\infty < \varepsilon$ .

- Note what is quantified:  $\mathcal{F}$  is the family with **unbounded** width, so “ $g$  exists” may mean a network of astronomical size
- Density says nothing about **how fast** the error falls as the network grows — that needs a **rate**

# The classical statement

*With the definition in place, the theorem is short.*

**Theorem 2 (Universal approximation; Cybenko 1989, Hornik 1991, Leshno et al. 1993)**

*Let  $h : \mathbb{R} \rightarrow \mathbb{R}$  be continuous. The family*

$$\mathcal{N}_{1,\infty} = \left\{ \mathbf{x} \mapsto \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)}) + w_0^{(2)} : M \in \mathbb{N} \right\}$$

*is dense in  $C(\mathcal{K})$  for every compact  $\mathcal{K}$  if and only if  $h$  is not a polynomial.*

- Proved in Class 7 by reduction to one dimension through **ridge functions**
- **One hidden layer is enough** — so universality alone can never be an argument for depth

How Wide Must a  
Network Be?

# Width as a threshold

*We know the cost of accuracy. Now: how to spend the budget. Width behaves quite unlike depth.*

## Theorem 3 (Width-bounded universality; Lu, Pu, Wang, Hu & Wang, 2017)

*ReLU networks of width  $D + 4$  and **arbitrary depth** are dense in  $L^1(\mathbb{R}^D)$ : for every Lebesgue-integrable  $f$  and every  $\varepsilon > 0$  there is such a network with  $\int |f - \hat{y}| \, d\mathbf{x} < \varepsilon$ .*

- Four units more than the input dimension is **all** the width that is ever needed, provided the network may be deep
- Note the norm: density in  $L^1$ , not uniform approximation

**Width is a threshold:** a small fixed margin above  $D$  is all that is ever needed. **Depth is a resource.**

How Much Does Depth  
Buy?

# The witness, recalled

*Width has a floor and then stops mattering. Depth has no such ceiling.*

## Lemma 4 (The sawtooth, from Class 7)

*$T(x) = 2h(x) - 4h(x - \frac{1}{2})$  is a two-unit ReLU network that maps  $[0, 1]$  onto  $[0, 1]$ , folding it in half. Its  $L$ -fold composition  $T^{\circ L}$  has exactly  $2^L$  linear pieces and is computed by a network of depth  $L$  and width 2.*

Two units per layer,  $L$  layers — and the number of pieces **doubles with every layer added**. That is the whole mechanism.

# Depth beats width, provably

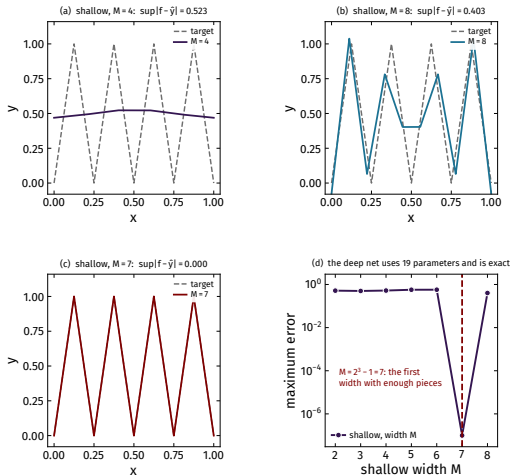
## Theorem 5 (Depth–width separation; Telgarsky, 2016)

$T^{\circ L}$  is computed by a ReLU network with  $O(L)$  parameters, but any *one-hidden-layer* ReLU network computing it exactly needs at least  $2^L - 1$  units. Telgarsky's full result strengthens “exactly” to “to within a constant”.

*Proof of the lower bound.* A one-hidden-layer ReLU network with  $M$  units on a scalar input is piecewise linear with at most  $M + 1$  pieces. Matching  $2^L$  pieces forces  $M + 1 \geq 2^L$ .  $\square$

An *exponential* gap, from a function you can draw in one line.

# The separation, measured



# A separation that is not one-dimensional

*The sawtooth lives on a line — easy to dismiss. The same gap appears in  $D$  dimensions.*

## Theorem 6 (Eldan & Shamir, 2016)

*There is a **radial** function  $g : \mathbb{R}^D \rightarrow \mathbb{R}$ , supported on a ball, that a network with **three** layers and  $\text{poly}(D)$  width approximates to within a constant in  $L^2$ , but which **every two-layer** network needs width  $\exp(\Omega(D))$  to approximate that well.*

- Why radial functions are hard for two layers: a two-layer net is a sum of **ridge** functions, each constant along  $D - 1$  directions, and building a sphere out of hyperplanes is expensive
- Three layers can first compute  $\|\mathbf{x}\|^2 = \sum_i x_i^2$  with  $D$  cheap units, then apply a one-dimensional function to it
- Unlike Telgarsky's, this separation is about **approximation in  $L^2$** , not exact representation, and grows with the **dimension** rather than the depth

# Counting Capacity

# Counting linear regions

*Rates are one currency; counting the pieces is another, and easier to compute.*

## Theorem 7 (Lower bound; Montúfar, Pascanu, Cho & Bengio, 2014)

*A ReLU network with  $L$  hidden layers of width  $M \geq D$  on  $\mathbb{R}^D$  can realise at least*

$$\left( \prod_{l=1}^{L-1} \lfloor M/D \rfloor^D \right) \sum_{j=0}^D \binom{M}{j}$$

*linear regions.*

The trailing  $\sum_{j \leq D} \binom{M}{j}$  is the **Zaslavsky count** from Class 5: the last layer behaves like a shallow network, and the product in front is what depth adds.

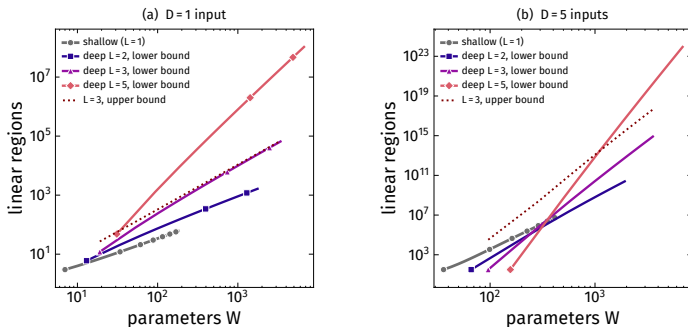
# Counting linear regions: the ceiling

## Theorem 8 (Upper bound; Serra, Tjandraatmadja & Ramalingam, 2018)

*The number of regions is at most  $\prod_{l=1}^L \sum_{j=0}^D \binom{M}{j}$ , and tighter bounds follow from the dimensions of the individual layers.*

- Both bounds are **exponential in  $L$**  and only **polynomial in  $M$**  — the same asymmetry as the sawtooth
- A lower bound says such networks **exist**; it does not say training finds them

# Regions against parameters



Regions against the parameter count  $W$ , for (a) a scalar input and (b)  $D = 5$ . Grey is the shallow count; the coloured curves are the Montúfar lower bound at three depths; dotted is the product upper bound at  $L = 3$ .

Widths are taken in multiples of  $D$ , where the lower bound is stated.

# Capacity has a price

*Every result so far has been an argument for depth. Here is the bill.*

## Theorem 9 (VC dimension; Bartlett, Harvey, Liaw & Mehrabian, 2019)

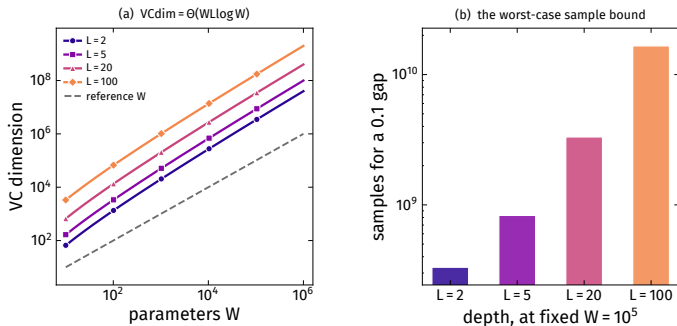
*For ReLU networks with  $W$  parameters and  $L$  layers,*

$$\text{VCdim} = \Theta(WL \log W).$$

- Learning theory then needs  $N = \Omega(\text{VCdim}/\epsilon^2)$  samples for a gap  $\epsilon$
- Modern networks have  $\text{VCdim} \gg N$  and generalise anyway: the bound is **loose** for real data (Class 17)

Expressivity and generalisation theorems pull in **opposite directions**; neither alone explains deep learning.

# VC dimension, drawn



(a)  $\Theta(WL \log W)$  against the parameter count, at four depths, with a linear reference. (b) at a fixed  $W = 10^5$ , the worst-case number of samples the VC bound demands for a generalisation gap of 0.1 — growing linearly with depth.

# Reading the Theorems

# What each theorem actually gives

*Ten results in one lecture. What each one is really claiming:*

| Result                    | Gives                                                 | Does not give                          |
|---------------------------|-------------------------------------------------------|----------------------------------------|
| Cybenko / Hornik / Leshno | density with one hidden layer, any non-polynomial $h$ | any bound on the width                 |
| Lu et al. (2017)          | width $D+4$ suffices, given depth                     | efficiency at that width               |
| Telgarsky (2016)          | an explicit exponential depth gap                     | that such functions matter in practice |
| Montúfar (2014)           | regions exponential in $L$                            | that training reaches them             |
| Bartlett et al. (2019)    | $\text{VCdim} = \Theta(WL \log W)$                    | a useful bound at modern scale         |

# What none of them say

*And the gap between all this and a working network:*

- **Nothing about optimisation.** Every result is about the *existence* of weights; finding them is Classes 11–14
- **Nothing about generalisation.** The capacity bounds are worst-case over all data distributions
- **The hard cases may be irrelevant.** The sawtooth and the radial bump are built to be hard; natural signals are not adversarial
- **Architecture matters more than the counts suggest.** Convolution and attention change the function class itself

**SHALLOW** universal, but with a vacuous width bound

**DEEP** universal *and* efficient, on the functions we care about

# Takeaways

1. Fixed basis functions need  $S^D$  parameters. Networks avoid this by placing their features, not tiling the space
2. One hidden layer is already universal, for any non-polynomial activation — so universality alone never argues for depth
3. A grid of fixed features costs  $S^D$  parameters; a network instead places its features where the data is
4. Width is a threshold ( $D + 4$  suffices, given depth); depth is a resource — Telgarsky and Eldan–Shamir give explicit exponential gaps
5. Capacity counts both ways: regions grow exponentially in  $L$ , and so does the worst-case VC bound

Class 11: having established what these networks can represent, how do we actually train them?

# References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §6.1 (limitations of fixed basis functions, the curse of dimensionality), §6.2 (multilayer networks), §6.3 (deep networks).
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 4 — composition, folding, region counting, parameters versus regions.
- G. Cybenko (1989); K. Hornik (1991); M. Leshno, V. Lin, A. Pinkus & S. Schocken, *Neural Networks* 6:861 (1993).
- Z. Lu, H. Pu, F. Wang, Z. Hu & L. Wang, *NeurIPS* (2017) — width-bounded universality.
- M. Telgarsky, *COLT* (2016); R. Eldan & O. Shamir, *COLT* (2016) — depth separations.
- G. Montúfar et al., *NeurIPS* (2014); T. Serra, C. Tjandraatmadja & S. Ramalingam, *ICML* (2018) — linear regions.
- P. Bartlett, N. Harvey, C. Liaw & A. Mehrabian, *JMLR* 20:63 (2019) — VC dimension.
- Plots generated by `../B_Recreated_Figures/src/build_all.py`.
- This variant carries only results stated or cited in the two textbooks. For Barron (1993), DeVore et al. (1989), Yarotsky (2017) and the width lower bound, see variant A or B.

Questions?